CS3423 Compilers II
Mini-Assignment 4: Clang AST Passes

CS24BTECH11030, CS24BTECH11036

October 2026

Contributions. CS24BTECH11030: Part 1 (AST dump and node analysis) and the traversal mechanisms section. CS24BTECH11036: Part 2 (the three AST-matcher passes, their tests and write-up).

1 Part 1: Exploring AST dumps

1.1 Program

```
int f(int x, int& y) {
  return x * x + y * y;
}

int main() {
  int k = 10, sum = 0;

  for (int i = 0; i < k; i++) {
    for (int j = 0; j < k; j++) {
      sum += f(i, j);
    }
  }

  int cnt = 0;
  while (sum != 1) {
    if (sum % 2) {
      sum = 3 * sum + 1;
    } else {
      sum /= 2;
    }

    cnt++;
  }

  return cnt;
}
```

1.2 Dumping the AST

```
clang -Xclang -ast-dump -fno-color-diagnostics -fsyntax-only nested_loops.cpp > ast_dump.txt
```

Running the above command dumps the AST into a local txt file. Below is a reduced version of the AST dump of `nested_loops.cpp`

```
TranslationUnitDecl 0x6239bf35e318 <<invalid sloc>> <invalid sloc>
|-TypedefDecl 0x6239bf35eb88 <<invalid sloc>> <invalid sloc> implicit __int128_t '__int128'
| `-BuiltinType 0x6239bf35e8e0 '__int128'
| ...
|-FunctionDecl 0x6239bf3a7eb8 <nested_loops.cpp:1:1, line:3:1> line:1:5 used f 'int (int, int &)'
| ...
`-FunctionDecl 0x6239bf3a8178 <line:5:1, line:26:1> line:5:5 main 'int ()'
  `-CompoundStmt 0x6239bf3c1ee8 <col:12, line:26:1>
    | ...
    |-ForStmt 0x6239bf3a88b8 <line:8:3, line:12:3>
    | |-DeclStmt 0x6239bf3a8460 <line:8:8, col:17>
    | | `-VarDecl 0x6239bf3a83d8 <col:8, col:16> col:12 used i 'int' cinit
    | |   `-IntegerLiteral 0x6239bf3a8440 <col:16> 'int' 0
    | |-<<<NULL>>>
    | |-BinaryOperator 0x6239bf3a84e8 <col:19, col:23> 'bool' '<'
    | | |-ImplicitCastExpr 0x6239bf3a84b8 <col:19> 'int' <LValueToRValue>
    | | | `-DeclRefExpr 0x6239bf3a8478 <col:19> 'int' lvalue Var 0x6239bf3a83d8 'i' 'int'
    | | `-ImplicitCastExpr 0x6239bf3a84d0 <col:23> 'int' <LValueToRValue>
    | |   `-DeclRefExpr 0x6239bf3a8498 <col:23> 'int' lvalue Var 0x6239bf3a8268 'k' 'int'
    | |-UnaryOperator 0x6239bf3a8528 <col:26, col:27> 'int' postfix '++'
    | | `-DeclRefExpr 0x6239bf3a8508 <col:26> 'int' lvalue Var 0x6239bf3a83d8 'i' 'int'
    | `-CompoundStmt 0x6239bf3a88a0 <col:31, line:12:3>
    |   `-ForStmt 0x6239bf3a8868 <line:9:5, line:11:5>
    |     |-DeclStmt 0x6239bf3a85e0 <line:9:10, col:19>
    |     | `-VarDecl 0x6239bf3a8558 <col:10, col:18> col:14 used j 'int' cinit
    |     |   `-IntegerLiteral 0x6239bf3a85c0 <col:18> 'int' 0
    |     |-<<<NULL>>>
    |     |-BinaryOperator 0x6239bf3a8668 <col:21, col:25> 'bool' '<'
    |     | |-ImplicitCastExpr 0x6239bf3a8638 <col:21> 'int' <LValueToRValue>
    |     | | `-DeclRefExpr 0x6239bf3a85f8 <col:21> 'int' lvalue Var 0x6239bf3a8558 'j' 'int'
    |     | `-ImplicitCastExpr 0x6239bf3a8650 <col:25> 'int' <LValueToRValue>
    |     |   `-DeclRefExpr 0x6239bf3a8618 <col:25> 'int' lvalue Var 0x6239bf3a8268 'k' 'int'
    |     |-UnaryOperator 0x6239bf3a86a8 <col:28, col:29> 'int' postfix '++'
    |     | `-DeclRefExpr 0x6239bf3a8688 <col:28> 'int' lvalue Var 0x6239bf3a8558 'j' 'int'
    |     `-CompoundStmt 0x6239bf3a8850 <col:33, line:11:5>
    |       `-CompoundAssignOperator 0x6239bf3a8820 <line:10:7, col:20> 'int' lvalue '+=' ComputeLHSTy='int'
                ComputeResultTy='int'
    |         |-DeclRefExpr 0x6239bf3a86c0 <col:7> 'int' lvalue Var 0x6239bf3a8308 'sum' 'int'
    |         `-CallExpr 0x6239bf3a87d8 <col:14, col:20> 'int'
    |           |-ImplicitCastExpr 0x6239bf3a87c0 <col:14> 'int (*)(int, int &)' <FunctionToPointerDecay>
    |           | `-DeclRefExpr 0x6239bf3a8768 <col:14> 'int (int, int &)' lvalue Function 0x6239bf3a7eb8 'f'
              'int (int, int &)'
    |           |-ImplicitCastExpr 0x6239bf3a8808 <col:16> 'int' <LValueToRValue>
    |           | `-DeclRefExpr 0x6239bf3a8728 <col:16> 'int' lvalue Var 0x6239bf3a83d8 'i' 'int'
    |           `-DeclRefExpr 0x6239bf3a8748 <col:19> 'int' lvalue Var 0x6239bf3a8558 'j' 'int'
    | ...
    |-WhileStmt 0x6239bf3c1e80 <line:15:3, line:23:3>
    | |-BinaryOperator 0x6239bf3a8a00 <line:15:10, col:17> 'bool' '!='
    | | |-ImplicitCastExpr 0x6239bf3a89e8 <col:10> 'int' <LValueToRValue>
    | | | `-DeclRefExpr 0x6239bf3a89a8 <col:10> 'int' lvalue Var 0x6239bf3a8308 'sum' 'int'
    | | `-IntegerLiteral 0x6239bf3a89c8 <col:17> 'int' 1
    | `-CompoundStmt 0x6239bf3c1e60 <col:20, line:23:3>
    |   |-IfStmt 0x6239bf3c1df8 <line:16:5, line:20:5> has_else
    |   | |-ImplicitCastExpr 0x6239bf3a8a98 <line:16:9, col:15> 'bool' <IntegralToBoolean>
    |   | | `-BinaryOperator 0x6239bf3a8a78 <col:9, col:15> 'int' '%'
    |   | |   |-ImplicitCastExpr 0x6239bf3a8a60 <col:9> 'int' <LValueToRValue>
    |   | |   | `-DeclRefExpr 0x6239bf3a8a20 <col:9> 'int' lvalue Var 0x6239bf3a8308 'sum' 'int'
    |   | |   `-IntegerLiteral 0x6239bf3a8a40 <col:15> 'int' 2
    |   | |-CompoundStmt 0x6239bf3a8ba8 <col:18, line:18:5>
    |   | | `-BinaryOperator 0x6239bf3a8b88 <line:17:7, col:23> 'int' lvalue '='
    |   | |   ...
    |   | `-CompoundStmt 0x6239bf3c1de0 <line:18:12, line:20:5>
    |   |   `-CompoundAssignOperator 0x6239bf3c1db0 <line:19:7, col:14> 'int' lvalue '/=' ComputeLHSTy='int'
                ComputeResultTy='int'
    |   |     ...
    |   `-UnaryOperator 0x6239bf3c1e48 <line:22:5, col:8> 'int' postfix '++'
    |     `-DeclRefExpr 0x6239bf3c1e28 <col:5> 'int' lvalue Var 0x6239bf3a8908 'cnt' 'int'
    `-ReturnStmt 0x6239bf3c1ed8 <line:25:3, col:10>
      `-ImplicitCastExpr 0x6239bf3c1ec0 <col:10> 'int' <LValueToRValue>
        `-DeclRefExpr 0x6239bf3c1ea0 <col:10> 'int' lvalue Var 0x6239bf3a8908 'cnt' 'int'
```

1.3 Interesting nodes and their hierarchy

1. `ForStmt`**.**

A `for` loop is one node with four fixed child slots, in order: *init* (`DeclStmt`), *condition variable* (here `<<<NULL>>>`, used only for `for (...; T v = ...; )` forms), *condition* (`BinaryOperator <`), *increment* (`UnaryOperator ++`), and finally the *body*. In the dump the body is a `CompoundStmt`, whose only child is the inner `ForStmt`. This is how loop nesting appears in the tree: the inner loop is a child of the outer loop's body.

2. `CallExpr`**.**

`f(i, j)` is denoted by `CallExpr` in the clang AST. Further, Its children are *function pointer* (Here achieved via `ImplicitCastExpr <FunctionToPointerDecay>`), and *parameters*. Another interesting thing which can be seen is the difference in how *CallByValue* and *CallByReference* affects the parameters passes. *CallbyValue* requires an `ImplicitCastExpr <LValueToRValue>`, while for *CallByReference*, we can directly pass in a reference `DeclRefExpr`.

3. `WhileStmt`**.**

A `while` loop has two child slots, in order: *condition* (Here a `BinaryOperator`), and *body* (`CompoundStmt`)

Other nodes worth noting.

`ImplicitCastExpr <LValueToRValue>` appears around every variable that is *read* (`i` and `k` in `i < k`) but not around `i` in `i++`, where it is modified in place.

`CompoundStmt` is the { ... } block and just holds a list of statements.

The overall root of this dump is the `TranslationUnitDecl`.

2 Traversal mechanisms in Clang

`RecursiveASTVisitor`**.** A CRTP base class. Derive from `RecursiveASTVisitor<Derived>` and define `VisitXxx(Xxx *node)` for the node kinds of interest; the base class walks the whole tree depth-first and calls them. Three layers exist per node kind: `TraverseXxx` (controls descent into children and can prune a subtree by not calling the base), `WalkUpFromXxx` (calls the `Visit` methods for the node and its base classes), and `VisitXxx` (what you usually override; return `false` to stop the whole traversal). Best for imperative analyses that need state across the walk.

AST matchers with `MatchFinder`**.** A declarative approach: describe the pattern as a composition of matchers (node matchers such as `forStmt`, `varDecl`; narrowing matchers such as `hasName`; traversal matchers such as `hasBody`, `hasCondition`, `hasDescendant`) and attach a `MatchCallback` to it with `MatchFinder::addMatcher`. `.bind("id")` labels sub-nodes, which the callback's `run` retrieves with `Result.Nodes.getNodeAs<T>("id")`. The finder does the walk internally. Concise for local patterns, and what Part 2 uses.

`ASTConsumer` **and** `ASTFrontendAction`**.** The hook by which a tool gets the AST at all. A `FrontendAction` creates an `ASTConsumer`, whose `HandleTranslationUnit(ASTContext&)` (or `HandleTopLevelDecl`) is called once parsing finishes. From there one typically starts a `RecursiveASTVisitor` from `Context.getTranslationUnitDecl()`. `newFrontendActionFactory(&Finder)` in a LibTooling `main` hides this for matchers.

`StmtVisitor / DeclVisitor.` Dispatch-only visitors: `Visit(node)` calls the `VisitXxx` for that node's dynamic type, but *does not* recurse; you recurse manually via `node->children()`. Used inside Clang itself (e.g. the AST printer, code generation) when the traversal order is custom.

Direct child iteration and parent queries. Every `Stmt` exposes `children()`, and a `DeclContext` exposes `decls()`, for a hand-written walk. Clang nodes have no parent pointer; `ASTContext::getParents(node)` (backed by the `ParentMapContext`) computes the parent map on demand for upward navigation.

3 Part 2: AST-matcher passes

All three checks are in a single LibTooling tool built from `pass.cpp` (target `Simple_Pass`), run as `Simple_Pass file.cpp --`. Each check has a test input in `tests/`.

3.1 Build and run

```
cp -r 2-ast-matcher-passes ~/llvm-project/clang-tools-extra/Mypass
# add:  add_subdirectory(Mypass)  to clang-tools-extra/CMakeLists.txt
cd ~/llvm-project/build && cmake . && make -j4 Simple_Pass
bin/Simple_Pass ../clang-tools-extra/Mypass/tests/assign_in_condition.cpp --
```

Tested against LLVM/Clang 17.0.6.

3.2 Pass 1: assignment inside a condition

Goal. Flag `if (x = 5)`, where `==` was probably intended.
Approach. The five statement kinds that have a condition (`if`, `for`, `while`, `do-while` and the `?:` operator) each get a matcher of the form shown below (here for `if`), and the five are combined with `anyOf` under one `stmt(..., isExpansionInMainFile())`.

```
ifStmt(hasCondition(ignoringParenImpCasts(
    binaryOperator(hasOperatorName("=")).bind("assignment"))))
```

`ignoringParenImpCasts` is needed because in C++ the condition `n = 13` is wrapped in an `ImplicitCastExpr` (int to bool), and `if ((n = 13))` adds a `ParenExpr`; both forms are therefore found. Only plain `=` is matched; compound assignments such as `+=` are not reported. The bind ID is placed on the `binaryOperator`, so the callback receives the assignment itself and prints its source location.
Test. `tests/assign_in_condition.cpp`
Output. Only the assignment on line 5 is reported; the comparison `n == 10` on line 8 is not.

```
tests/assign_in_condition.cpp:5:8
```

3.3 Pass 2: empty loop body

Goal. Flag `for`, `while` and `do-while` loops whose body is an empty compound statement {}.
Approach. `forStmt`, `whileStmt` and `doStmt` each use `hasBody(compoundStmt(statementCountIs(0)))`, i.e. the body must be a block containing no statements, and all three bind the same ID `loop`. A loop whose body is a single statement or

a bare `;` (`NullStmt`, as in `for (...);`) is deliberately not reported, since the question asks for empty compound statements. The matchers are restricted to the main file.
Test. `tests/empty_loop_body.cpp`
Output. Only the empty `for` on line 6 is reported; the `do` and `while` loops in the test have non-empty bodies.

```
tests/empty_loop_body.cpp:6:5
```

3.4 Pass 3: unused local variable

Goal. Flag local variables that are declared but never referenced in their enclosing function.
Approach. Two matchers feed one callback: `varDecl(hasLocalStorage(), unless(parmVarDecl()))` records every local variable declaration, and `declRefExpr(to(varDecl(hasLocalStorage())))` records every use. Each variable has exactly one `VarDecl` node, and each use is a `DeclRefExpr` whose `getDecl()` points back to it, so the `VarDecl*` identifies the variable (two variables both called `i` are different pointers). The callback keeps a set of declared and a set of referenced pointers; after `Tool.run` the variables in the first set but not in the second are printed. The report cannot be produced inside the callback because a use may appear after the declaration. Parameters, globals and `static` locals are not reported. Any reference counts as a use, so a variable that is only written (`int c = 3; c = 4;`) is not reported.
Test. `tests/unused_local.cpp`
Output. The unused `x` in `f` and the unused `x` in `main` are both reported; `n` and `i` are used and are not.

```
Unused variable: x
Unused variable: x
```

4 Limitations

- Pass 1 reports only plain `=`; `if (x += 2)` is not reported. An assignment hidden in a function call or a lambda inside the condition is not found, because the matcher inspects only the condition expression itself.
- Pass 2 does not report a `for (...);` or `while (...);` loop (a `NullStmt` body) or a body containing only an empty nested block.
- Pass 3 counts any reference as a use, so write-only variables are missed; it does not report unused parameters, globals or `static` locals, and the report prints only the variable name (no source location).
- All passes ignore code that comes from headers (`isExpansionInMainFile`), so problems inside included files are not reported. Macros and templates were not tested.

Note on AI use

The text of this write-up was drafted with AI assistance (Claude). The passes themselves were written by the authors.